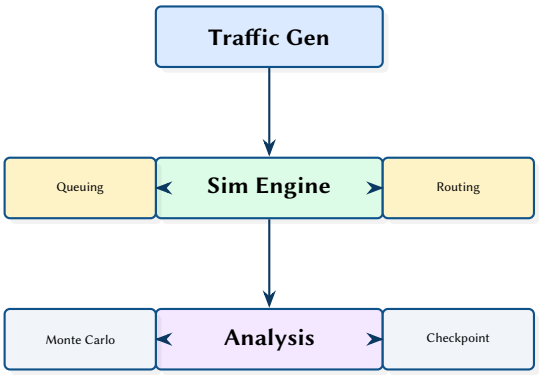
# NetFlowSim: A Discrete Network Flow Simulator with Queuing Models and Monte Carlo Analysis

## Traffic Generation, Routing, Failure Cascades, and Checkpoint/Restore

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 11, 2026




**Abstract.** NetFlowSim is a Rust-based discrete network flow simulator that models traffic flows across graph topologies with pluggable queuing models (M/M/1, M/D/1, M/M/c, M/G/1), configurable service time distributions (Exponential, Pareto, LogNormal, Weibull), and weighted multi-path routing. The simulator supports Monte Carlo analysis with parallel execution via Rayon, deterministic seeding for reproducibility, and binary checkpoint/restore for resumable long-running simulations. An analysis framework provides N-1 resilience testing, bottleneck detection, capacity planning, cascade failure modelling, correlation analysis, and regional traffic analytics.

# Contents

# 1 Introduction and Motivation

Understanding network behaviour under load requires simulation that captures both the topology structure and the queuing dynamics at each node. NetFlowSim combines a graph-based network model with classical queuing theory to simulate traffic flows, measure per-link utilisation, and identify failure modes through systematic analysis.

In practical settings, capacity problems rarely present as a single overloaded link. Instead, they emerge from interactions between traffic mix (elephant vs. mice flows), routing decisions (equal-cost or weighted multi-path), stochastic service times, and failure-driven reconvergence. A simulator that treats the network as a static graph with deterministic link delays can miss these second-order effects. NetFlowSim is designed to expose them by combining:

- **Discrete flow simulation** over arbitrary graph topologies.
- **Queueing-theoretic delay models** at nodes/links to represent contention and variability.
- **Stochastic workload models** through configurable service time distributions.
- **Repeatable Monte Carlo analysis** to quantify uncertainty.
- **Operational analyses** (resilience, bottlenecks, cascades) that connect simulation outputs to engineering decisions.

The remainder of this report documents the architecture of NetFlowSim, the modelling assumptions it makes, and the design decisions that prioritise reproducibility, extensibility, and performance.

# 2 Simulation Engine

The engine is built on a `petgraph::StableGraph` [1] with indexed simulation for performance. Nodes and links can be dynamically disabled/enabled to model failures, and a routing cache avoids recomputation when the topology is unchanged.

## 2.1 Core Data Model

At runtime, the simulator maintains three interlocking state families:

- **Topology state** (nodes/links, capacities, enable flags).
- **Flow state** (source, destination, demand/arrival process, current route choice, and per-flow measurements).
- **Statistics state** (per-link utilisation, queueing delay aggregates, loss/overload counters, and per-region summaries).

The `StableGraph` choice ensures node and edge indices remain valid when elements are disabled, which is crucial for cached routing and for checkpoint/restore fidelity.

```rust
pub struct NetworkSimulator {
    graph: StableGraph<NodeState, LinkState>,
    routing_matrix: Option<RoutingMatrix>,
    flows: Vec<Flow>,
    rng: ChaCha8Rng,
    stats: SimulationStats,
}
```

## 2.2 Queuing Models

Four queuing models are available, selected per-node:

**Table 1.** Supported queuing models.

| Model | Parameters | Delay Formula |
|---|---|---|
| M/M/1 | $\lambda, \mu$ | $\frac{1}{\mu-\lambda}$ |
| M/D/1 | $\lambda, \mu$ | $\frac{2-\rho}{2\mu(1-\rho)}$ |
| M/M/c [2] | $\lambda, \mu, c$ | Erlang-C based |
| M/G/1 [3] | $\lambda, \mu, C_s^2$ | Pollaczek–Khinchine |

The M/G/1 model accepts arbitrary service time distributions through a trait-based design:

```rust
pub trait ServiceDistribution: Send + Sync {
    fn mean(&self) -> f64;
    fn cv_squared(&self) -> f64;  // Coefficient of variation squared
```

```
    fn sample(&self, rng: &mut impl Rng) -> f64;
    fn cdf(&self, x: f64) -> f64;
}
```

Implementations include Exponential, Deterministic, Pareto, LogNormal, and Weibull distributions.

### 2.3 Time and Load Interpretation

NetFlowSim uses queueing models to map an offered load to an expected delay. For a given service rate $\mu$ and offered arrival rate $\lambda$, utilisation is $\rho = \lambda/\mu$. The selected queueing model then computes a mean waiting/sojourn time contribution. This abstraction is useful for two reasons:

- It supports both **deterministic** and **variable** service times without changing the simulator core.
- It enables **sensitivity studies**: the same topology and routing can be evaluated under different $C_s^2$ (service variability) assumptions to understand tail risk.

When $\rho \to 1$, delays increase sharply; the analysis modules treat these regimes as early indicators for congestion-driven cascades.

> **Design Decision 1: Pluggable Service Distributions**
>
> By parameterising queuing models with a `ServiceDistribution` trait rather than hardcoding distribution types, NetFlowSim can model both the heavy-tailed traffic patterns typical of internet workloads (Pareto) and the deterministic processing times of hardware forwarding engines.

## 3 Traffic Generation

The traffic generator produces flows with configurable source/destination selection. A region-aware locality bias ensures that a configurable fraction of traffic stays within the same region, modelling realistic enterprise traffic patterns.

### 3.1 Flow Demand and Arrival Processes

Traffic generation is modelled as a set of flows with configurable intensity. Depending on the scenario, flows can represent:

- **Aggregated demands** between endpoints (e.g., site-to-site).
- **Microflows** (e.g., application requests) whose aggregate arrival rate produces load on shared resources.

Locality bias is implemented at selection time: a portion of source/destination pairs are sampled within the same region, with the remainder sampled globally. This allows direct evaluation of regionalisation decisions (e.g., shifting traffic to stay local) and their impact on backbone utilisation.

### 3.2 Routing

The routing module generates a full routing matrix using weighted shortest paths. Multi-path routing distributes flows across Equal-Cost Multi-Path (ECMP) paths with configurable load-balancing weights.

#### 3.2.1 Routing Cache and Failure Sensitivity

Routing computation can dominate runtime when repeatedly simulating the same topology under different traffic seeds. NetFlowSim therefore caches the routing matrix and invalidates it when a topology mutation occurs (e.g., link down). This is paired with the scheduled event system (Section 6) to model reconvergence: an event that disables a link both changes the feasible path set and may concentrate load onto the remaining paths.

Weighted multi-path routing is used to emulate common operational strategies: prefer high-capacity links, avoid expensive peering edges, or intentionally split load to reduce variance. The analysis framework can then quantify whether such strategies reduce maximum utilisation or instead increase path stretch and latency.

## 4 Analysis Framework

Six analysis modules operate on simulation results:

Table 2. Analysis modules.

| Module | Description |
|---|---|
| N-1 Resilience | Tests every single-element failure; reports impact |
| Bottleneck | Identifies links/nodes with highest utilisation |
| Capacity Planning | Projects when links will reach capacity under growth |
| Cascade Failure | Models progressive failure under overload |
| Correlation | Finds correlated failure patterns |
| Regional | Per-region traffic flow analysis |

### 4.1 Monte Carlo Simulation

The Monte Carlo module runs parallel iterations using Rayon [4] with deterministic per-iteration seeding:

```
pub fn run_monte_carlo(
    config: &MonteCarloConfig,
    topology: &Topology,
) -> MonteCarloResults {
    (0..config.iterations)
        .into_par_iter()
        .map(|i| {
            let seed = config.base_seed ^ i.wrapping_mul(0x9E3779B97F4A7C15);
            let mut sim = NetworkSimulator::new(topology.clone(), seed);
            sim.run_iteration(&config.scenario)
        })
        .collect()
}
```

**: Deterministic Monte Carlo**

Each iteration derives its Random Number Generator (RNG) seed deterministically from the base seed and iteration index. Results are identical regardless of Rayon's thread scheduling, enabling reproducible analysis.

### 4.2 Outputs and Metrics

Simulation outputs are structured to support both engineering investigation and statistical summarisation. Common metrics include:

- **Per-link utilisation**: mean and peak $\rho$ values.
- **Queueing delay**: mean delay and variability proxies.
- **Path stretch**: routing cost relative to baseline shortest path.
- **Failure impact**: incremental congestion and disconnected demand under N-1 scenarios.
- **Regional flows**: intra- vs. inter-region demand matrices.

Monte Carlo runs report distributions (e.g., quantiles) rather than only point estimates, which helps avoid overfitting decisions to a single seed.

## 5 Checkpoint and Restore

Long-running simulations can be checkpointed to disk using binary serialisation (bincode). The checkpoint captures the full simulator state: topology, flow state, RNG seeds, routing matrix, and accumulated statistics.

### 5.1 Reproducibility and Compatibility

Checkpoint/restore is treated as an architectural feature, not merely a convenience. It enables:

- **Resumable experiments**: pause and continue long Monte Carlo campaigns.
- **Debuggable scenarios**: reload a problematic state to inspect the immediate pre-failure conditions.
- **Result provenance**: persist the exact state that produced an outlier metric.

***Insight:***
*Checkpoint files include a schema version field, enabling forward-compatible deserialisation as the simulator evolves. Unknown fields in newer schemas are silently ignored when loading older checkpoints.*

## 6 Scheduled Events

The event system supports time-scheduled link and node failures with configurable convergence modelling. Events can be defined as:

- **Deterministic** — Fixed time, fixed target
- **Stochastic** — Poisson-distributed failure times
- **Correlated** — Failures triggered by other failures (shared risk link groups)

### 6.1 Convergence Modelling

Operational networks typically exhibit transient behaviour after a failure: routing reconverges, load shifts, and queues may temporarily build even if the post-failure steady state is feasible. NetFlowSim captures this by allowing events to carry convergence semantics (e.g., apply immediately vs. apply after delay), enabling experiments that distinguish between:

- **Steady-state feasibility**: is there enough capacity after rerouting?
- **Transient risk**: do reconvergence dynamics create a window of overload large enough to trigger secondary failures?

## 7 Design Summary

This section consolidates architectural choices and their intended effects on usability and correctness.

**Table 3.** Key design decisions.

| Decision | Rationale |
|---|---|
| StableGraph | Node/link disable without index invalidation |
| Trait-based distributions | Pluggable service times for diverse workloads |
| ChaCha8Rng | Cross-platform deterministic simulation |
| Binary checkpoints | Fast serialisation for multi-hour simulations |
| Rayon parallelism | Near-linear Monte Carlo scaling across cores |

Beyond these decisions, a consistent theme is **separating concerns** between (i) topology and routing, (ii) load-to-delay modelling, and (iii) analysis. This separation makes it easier to extend the simulator with new queueing models or routing strategies without rewriting the analysis pipeline.

## References


[1] bluss, *Petgraph: Graph data structure library for Rust*, 2024. [Online]. Available: https://crates.io/crates/petgraph

[2] A. K. Erlang, *Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges*. 1917.

[3] F. Pollaczek, "Über eine aufgabe der wahrscheinlichkeitstheorie," *Mathematische Zeitschrift*, 1930.

[4] Rayon contributors, *Rayon: Data parallelism library for Rust*, 2024. [Online]. Available: https://crates.io/crates/rayon

## List of Figures

## List of Tables

## List of Design Decisions & Rules



## Revision History

| Version | Date | Changes |
|---|---|---|
| 0.1.0 | March 2026 | Initial architecture report |